# FABRID-IDS: Federated Alert-Budget Reallocation for Heterogeneous IoT Intrusion Detection

Salaheddine Naslouby [1*], Mohamed Lachgar [2] and Hamid Hrimech [1]

[1]Hassan First University, National School of Applied Sciences (ENSA), LAMSAD Laboratory, Berrechid, Morocco.
[2]Cadi Ayyad University, Higher Normal School (ENS), L2IS Laboratory, Marrakech, Morocco.

*Corresponding author(s). E-mail(s): naslouby.salahe@gmail.com;
Contributing authors: m.lachgar@uca.ac.ma; hamid.hrimech@uhp.ac.ma;

**Abstract**

High pooled rank performance can occur with anomaly scores that give bad per-client performance once thresholds are set. This paper looks at the distribution of a federation nominal FPR budget after the detector has been trained. Nominal client targets are optimized together and then turned into thresholds using a disjoint set of benign samples. The approach is tested on 9 N-BaIoT clients with 10 detector seeds and 5 budgets. A second setup with 19 CIC IoT-DIAD clients uses a different set of features and ranker. The two unrestricted allocation methods (Macro and Tail) achieved a heldout federation FPR of 0.00170 when B = 0.001. This is 1.696 times the budgeted FPR. Calibrating the methods more conservatively lowered the FPR to 0.00036 while sacrificing some macro recall (0.3778→ 0.2445). Equal-FPR gave better macro recall at some budgets but also overshot its budgeted FPR at some budgets. When looking at rank there are very high values for pooled AUROC and AUPRC of 0.9995 and 0.9999 respectively. However across all FABRID policies and budgets there was a client that had 0 recall for attack on N-BaIoT. This highlights a weakness in trying to distribute a nominal FPR across clients. It will not guarantee the federation FPR or detect attacks on every client.

**Keywords:** federated intrusion detection, Internet of Things, alert-budget reallocation, false-positive-rate constraint, threshold calibration

# 1 Introduction

After the detector is trained it gives an anomaly score. Once the score exceeds the threshold an alert will be sent. Deciding how to set this threshold can be hard when the clients that take part in federated learning have different benign score distributions. Or there could be variations in how attacks are scored. Federated learning for IoT Intrusion detection systems has been researched for both global and client specific thresholds [1–6]. What about choosing which clients to allocate a higher or lower nominal FPR if a budget is given to the federation?

This paper focuses on setting nominal client FPRs after the anomaly scores have been learned. It will use validation attacks and benign frontier scores to choose these client FPRs ($\alpha_k$). $\sum_k w_k \alpha_k \leq B$ will keep the allocation of FPRs below the desired budget B. Clients will then use a different set of benign samples to find their threshold for sending alerts. The held out test data can evaluate both the choice of nominal FPR as well as the ability to set these thresholds from a limited set of samples. Budget B will only affect how high the nominal FPR allocations can be set not the actual number of false positives that will occur.

How does this nominal FPR compare to what can be achieved when finding client thresholds using the calibration samples? All of the N-BaIoT allocation methods will be tested on the same anomaly scores and detector seeds.

# 2 Related work and scope

It is helpful to differentiate between how a detector is trained and what alert policy is applied when placing FABRID-IDS in context. Much of the literature on federated IoT IDS has focused on learning architectures and aggregation mechanisms [1–3, 7]. There are also works that look at anomaly thresholds and detecting at the edge [8, 9]. Laridi et al. [4] pool summary statistics from the client to determine an overall anomaly threshold. Asiri et al. [5] obtain local quantiles for the error of reconstructing benign traffic and pool those thresholds. Khan et al. [6] obtains a local threshold based on a holdout of benign data at a desired local FPR. These are all related thresholding approaches or offer useful context but none look at allocating the federation nominal FPR.

Restricting error rates can also be seen in Neyman-Pearson type classification problems [10]. Conformal methods use calibration ranks to provide statements about finite sample performance given some assumption about sampling [11]. There is research looking at sharing predictors and aggregating calibration scores in a federated setup but address a slightly different problem [12]. FABRID-IDS uses these calibration results as a black box and decides on the nominal targets before the thresholds are found. It focuses on allocating the nominal FPR across the federation and shows what that allocation produces.

There are other works on fairness at training time and agnostic federated learning which try to modify the loss function or effect of a client [7]. FABRID-IDS does not do this. It can alter the value of $\alpha_k$ after training is complete and this will effect the decision threshold for client k but will not change its ranking of benign or attack traffic. If the scorer does not perform well, all of the policies will suffer. Changing the nominal

rates does not allow detection of an attack that the scorer would always put under the threshold.

First a system can have a global absolute threshold and compare everything against that value. Second a system can allow local thresholds but specify a nominal FPR across all clients (Equal-FPR). Finally like Equal-FPR FABRID-IDS calibrates thresholds locally but can allocate the FPR based on the weighted constraint.

# 3 FABRID-IDS

## 3.1 Protocol and utility

The frozen scoring function for client k is denoted as $s_k(\cdot)$. The data is split into training 50%, find frontier 20%, final calibration 10% and test 20% using the file order given for the benign data. Each attack subtype will reserve its first 20% as validation with the rest as test. When allocating the nominal rates there is access to the frontier points and attack validation points. After allocation the final calibration points are used to determine a threshold and the test points are unused. These percentages are how the benchmark protocol was run and are not known to reflect real world splits of time.

Given a sample of size n of benign data in the calibration set. Let $x_{(1)} \leq ... \leq x_{(n)}$ be its order statistics. Given a nominal rate alpha it will be mapped to a threshold by equation 1 [11].

$$\tau(\alpha) = x_{(r)}, \qquad r = \lceil (n+1)(1-\alpha) \rceil, \tag{1}$$

When alpha $\leq 0$ tau = infinity or when n = 0 or r is greater than n. An alert will occur if the score is greater than tau. This is applied to the frontier points and nominal rate to find a temporary threshold. Then $u_k(\alpha)$ is computed which is the mean recall of each valid attack subtype for client k. Attack labels are required to create this function. The search space for alpha is a grid of 207 values from 0 to .05 and includes the desired budgets. Subtypes must have 50 validation points to be considered and a client needs 200 validation points across at least 2 subtypes.

When finding model parameters the training split of benign points for each client is used. Once an allocation has been chosen the thresholds from the frontier are thrown out and Eq. 1 is used on the final calibration set. Test points are never used. While the frontier points are not being used to find a final threshold it does not mean that they have the same distribution as the calibration and test points.

The nominal rates will be closer together than what is possible from the sample of benign points. Choosing multiple $\alpha_k$'s that lead to the same value for $x_r$ is technically allocating them different rates. But this will get the same result on the test set as the scoring function is static. So any change in nominal rates should be interpreted as doing something different only if it leads to a change in the threshold.

To find the recall for attacks it is first computed for each valid subtype and the mean is taken on the client level. The endpoint of federation macro recall is the mean of all the valid clients. And the last endpoint is the minimum value of the client level means. The FPR on the hold out set of benign points will be reported as well.

### 3.2 Allocation and calibration

The nominal rates will be found from the given grid of candidates. The choices must satisfy that the sum of the weights times each nominal rate is less than or equal to the budget.

$$\sum_{k=1}^{K} w_k \alpha_k \leq B. \tag{2}$$

Where $w_k = 1/\mathrm{K}$ for N-BaIoT so the budget is an average across the clients. This is solved as a mixed integer problem by having the allocator pick one alpha value for each client. The HiGHS library is used to solve for the Macro and Tail objective functions. Macro tries to maximize the average of $u_k$. Tail will maximize the mean of the bottom q = ceil(0.25K) values of $u_k$ [13]. As there are 9 devices q=3. It is similar to empirical CVaR but will have some numerical differences as it is taking the average of the bottom three values instead of applying a fractional weight to the third value [13]. Each point in the frontier will have some minimal FPR they can generate. So if $n_f$ is the number of frontier points for the benign class all positive candidate alpha values that are less than $2/(n_f + 1)$ will be discarded. Since they can't generate a positive FPR when calibrating.

Do not confuse this definition of equal with using all the benign points and finding a global FPR. Eq. 2 states that one can not exceed the budget. One can choose to underutilize the budget and assign a client 0 if the utility gained is not worth the trade-off of increasing their nominal rate.

For each problem there will be a list of criteria to optimize for. For Macro the mean estimated utility is first optimized. Then the nominal amount spent is minimized subject to the solver's notion of optimality. For Tail the mean of the bottom three utilities is optimized. Then optimize for the mean estimated utility and finally minimize the nominal spending. The variance of the estimated utilities and then some form of deterministic tie break will be used to find the solution. Note only the frontier points and validation attacks are being used to find these allocations. Equal-FPR simply gives each client a value of B. All the unrestricted policies will share the same benign points and Eq. 1. To find the final thresholds for the conservative Macro and Tail Eq. 3 will be used.

One should note that by optimizing for a higher mean of the bottom three utilities does not mean the worst client will be protected better. Say the bottom three devices each have an estimated recall of 0.2. If two of those devices were increased by 0.1 and one left alone the objective would be improved but that one device could still fail to detect any attacks in the evaluation.

When looking for a more conservative way to calibrate, Tong et al. [10] give the rank needed to achieve a 95% confidence in the population FPR.

$$r_{.95} = \min\{r : \Pr[\mathrm{Binomial}(n, 1-\alpha) \leq r-1] \geq 0.95\}. \tag{3}$$

For each client it is possible to search from rank 1 to n until finding a value that meets the criteria. If the criteria is never reached or n = 0 or alpha = 0 there will again be no alerts by setting the threshold to infinity. Otherwise an alert occurs if s > tau.

If i.i.d. sampling is assumed for the calibration and future score values a one-sided 95% confidence statement can be made about the population FPR for each client [10]. It should be noted this is only about the sampling distribution and not that the test partition is known to be a sample from that distribution. This is evaluated as a secondary result to show that if one were to allocate in a federated manner and use a more conservative way to find thresholds it is possible to exceed the budget.

As the rank will likely be higher than Eq. 1 it may decrease the false alarms but may increase the false negatives. The test labels are not being used to make decisions on a threshold. So if there is different behavior in the benign scores between the calibration set and test set it will not be possible to correct for it.

If it is found that one of the clients exceeds their nominal FPR on a hold out set of data. That does not mean they violated the constraints when allocating. Or if one of the clients stays under their allocated FPR on a partition that does not mean it will stay under in the future. Which is why the allocated nominal FPR, rank on the final calibration set and the FPR observed on the hold out data are tracked. All the unrestricted policies will just vary in the nominal rates they assign to each client.

The minimal nonzero FPR on the calibration set of n points without any ties is 1/n. So any value below 2/(n+1) will give an r that is out of range or find the max of the points. So any rate below this threshold will give 0 alerts on the calibration set. This means that the grid of candidates will have some nominal rates that have no effect on the FPR. For the ecobee device there are 1,311 points so any rate below 0.001524 will produce no alerts during calibration. Again this may still give a positive FPR on the test set but no alerts during calibration.

### 3.3 Frozen scorer

For each device on N-BaIoT a hybrid scoring function is created using the reconstructed error from the AE and `HH_jit_L0.1_mean`($f_{75}$). The percentile of these two values is taken using a fixed reference from the frontier and the max is taken. The feature and scorer were chosen from development splits using predeclared development-only ablations and not modified when experimenting with different threshold policies. The AE had a hidden layer size of [64, 16] and was trained for 10 rounds of federated learning. Taking 3 local epochs in each round. Adam was used with a learning rate of 0.001 and a batch size of 64 and MSE loss. The clients were weighted by the amount of rows they had for training. And used the training split to normalize the clients. The detectors used 10 seeds and each policy will always use the same scores. Meaning AUROC and AUPRC can not be affected by modifying the threshold.

## 4 Experimental methodology

The N-BaIoT dataset is used to evaluate FABRID-IDS. Each of its 9 devices will be used as the clients and will be tested with budgets of B=0.001, 0.0025, 0.005, 0.010 and 0.020. The data will contain benign and BASHLITE and Mirai attacks [14]. The amount of data varies greatly per client with ecobee having 13,113 points of benign and philips baby monitor having 175,240. So when splitting the data to find the final threshold each device will have different amounts of precision. Macro recall is computed

on the subtypes of each client and then the mean of that value is found. The worst client is just the min of these values.

For each seed, all policies use the same detector and scoring data. So when comparing the results from FABRID-IDS Macro and Equal-FPR they are compared within that seed. There will be 10 total values from the 10 seeds and the mean can be found across those seeds. This mean is from the random initialization and training process not a mean over newly found clients or periods of time.

The same weights from Eq. 2 are used to compute the hold out federation FPR. Which means that while the allocation respects the nominal budget, the measured FPR may not match. Due to how the data is split and the windows used to create features. Training with different seeds will not solve this problem.

On N-BaIoT it was decided to compare Macro and Equal-FPR on federation recall and Tail and Equal-FPR on min client recall. The outcome is found on each seed and then Equal-FPR is subtracted. The data points are not replicated by having different amounts of data or attack types to detect on.

Using the paired sign flip test one can enumerate through all $2^{10}$ ways of assigning the 10 values to a negative or positive. To create the confidence interval the bootstrap samples with replacement from those values 50,000 times, computes the mean for each sample, and finds the 2.5th and 97.5th percentiles of those means. The budgets are then p value adjusted with Holm. This gives a representation of how the detector changes with 10 different training scenarios. Not that it is possible to generalize this to new clients.

The CIC IoT-DIAD dataset is created by Rabbani et al. [15] while working on device identification and anomaly detection. This is used as a second example of the protocol. It has 19 devices that can be used as clients and 57 features. Similar to N-BaIoT a detector will be trained on this dataset. The benign split and % of attacks used for validation will be the same. No information or detector is carried over from N-BaIoT. As everything from data to client is different it can not be said what caused any discrepancy in FPR.

# 5 Results

## 5.1 Nominal and realized operating points

When B is set to 0.001 the unrestricted forms of Macro and Tail give the same realized federation FPR of 0.00170 (1.696B). If the more conservative version of these two is used the FPR drops to 0.00036 (0.358B) but the macro recall goes down to 0.2445 from 0.3778. The nominal FPR is kept at or below the set budget for all four methods. However the realized FPR is what results when looking at held out benign samples.

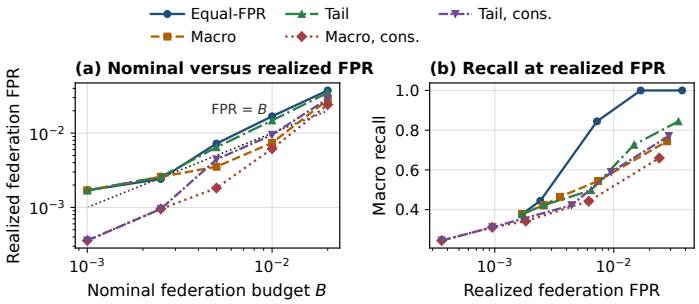



**Fig. 1** N-BaIoT ten-seed averages. Left shows realized FPR vs the nominal and right shows the macro recall vs realized FPR. Diagonal line shows equal rates.

**Table 1** N-BaIoT ten-seed averages. Cell shows the macro recall and the realized federation FPR / Nominal budget. FABRID family methods have a min client recall of zero.

| $B$ | Equal-FPR | Macro | Tail | Macro, cons. | Tail, cons. |
|---|---|---|---|---|---|
| 0.0010 | 0.3778; 1.70 | 0.3778; 1.70 | 0.3778; 1.70 | 0.2445; 0.36 | 0.2445; 0.36 |
| 0.0025 | 0.4445; 0.97 | 0.4222; 1.03 | 0.4222; 1.03 | 0.3111; 0.38 | 0.3111; 0.38 |
| 0.0050 | 0.8444; 1.45 | 0.4644; 0.71 | 0.4977; 1.29 | 0.3422; 0.36 | 0.4222; 0.88 |
| 0.0100 | 0.9999; 1.68 | 0.5444; 0.74 | 0.7266; 1.48 | 0.4422; 0.62 | 0.5889; 0.95 |
| 0.0200 | 1.0000; 1.88 | 0.7444; 1.40 | 0.8444; 1.75 | 0.6600; 1.21 | 0.7711; 1.45 |

Each of these methods does not behave identically when comparing the nominal to realized FPR. At B = 0.0025 both the Macro and Tail methods run at about 1.03B, slightly over their budget. Some budgets exceed the nominal FPR when using Equal-FPR so the difference between nominal and realized FPR isn't just caused by the optimal allocation method. These realized FPR values are plotted against the nominal in Figure 1(a) and it can be seen how far off the diagonal line of equal rates they fall. Figure 1(b) shows the realized macro-recall for each method against their measured FPR.

If the nominal budget is set to B = 0.005. The macro recall for Equal-FPR is 0.8444. Both the Macro and Tail methods have a recall of 0.4644 and 0.4977. When looking at the realized federation FPRs they were 1.45B, 0.71B and 1.29B respectively.

The variation of rate allocations was also looked at when resampling the frontier benign and validation attacks scores per subtype. The unrestricted form of Macro was taken and the scores were bootstrapped 500 times for each seed and budget. The train scorer or the test scores or the final calibration scores were not modified. For each client-seed-budget combination the percentage of rates that were different than the mode over the 500 bootstrap resamples was found. Averages over the 10 seeds and 9 clients give 77.8%, 79.8%, 83.9%, 83.6% and 84.9% respectively as the budget increases. This number is averaged over 90 seed-client frequencies so it is not telling the percentage of seeds that vary in rate. There is variation in the optimized decision given the amount of data on the frontier. Although these are not multiple instances of the system running nor does this describe how different they are from the original decision.

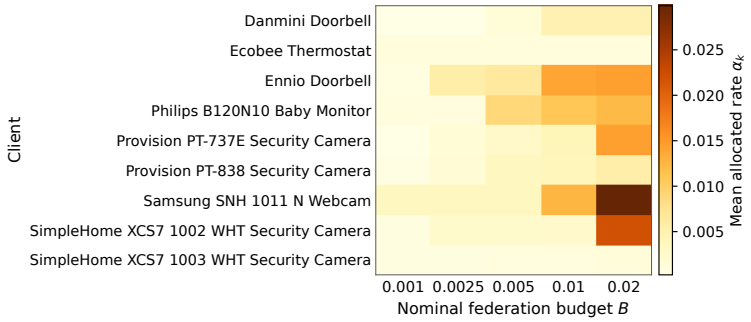



**Fig. 2** Average rates chosen by the unrestricted Macro method for the N-BaIoT devices. This is averaged over 10 seeds of the original detector. Bootstrap resamples of the frontier are not shown.

In Fig. 2, the Samsung webcam and SimpleHome XCS7 1002 security camera had the highest average rates when B = 0.020, whereas the Danmini doorbell was given a lower rate.

## 5.2 Tail objective and worst-client behavior

Although Tail looks at the mean of the lowest 3 client utilities. All methods that fall under the FABRID family showed a minimum-client recall of zero. Table 1 shows this for all budgets. There were no attacks detected on the Danmini Doorbell device. For all unrestricted-policy operating points tested. This is the cause of the minimum endpoint failing. There are also no alerts made on the Danmini's benign held-out data and final calibration data. All 100 unrestricted-policy-seed-budget combinations gave an FPR of zero for the Danmini. The Danmini final calibration benign data has a length of 4,955 so at threshold $2/(4{,}955+1) = 0.000404$ there is a transition from making zero alerts.

Rates are allocated that are both above and below this value. But a rate is never found that will detect attacks on this device when using unrestricted policy.

When looking at the ecobee device at B = 0.020 the macro method allocates an average nominal rate of 0.001164. The ecobee gets a held-out benign FPR of 0.006672 and achieves 0.6000 macro attack recall. Despite the ecobee having a final calibration FPR of zero on the benign data.

It is known that increasing the calibration sample size can increase the resolution of the rate threshold. But there could also be other factors causing the difference in alerts made on the test data. It is not known if these splits are i.i.d. so it cannot be said if there is a larger score distribution difference for some clients.

**Table 2** Seed level comparisons. Using 10 seeds a 50,000 bootstrap CI and p-value were computed using a sign flip test with Holm correction over the 5 budget family.

| | Macro − Equal-FPR: macro recall | | | Tail − Equal-FPR: minimum-client recall | | |
|---|---|---|---|---|---|---|
| $B$ | Mean Δ | 95% CI | Holm $p$ | Mean Δ | 95% CI | Holm $p$ |
| 0.0010 | 0.0000 | [0.0000, 0.0000] | 1.0000 | 0.0000 | [0.0000, 0.0000] | 1.0000 |
| 0.0025 | −0.0222 | [−0.0556, 0.0111] | 0.3594 | 0.0000 | [0.0000, 0.0000] | 1.0000 |
| 0.0050 | −0.3800 | [−0.4133, −0.3466] | 0.0098 | 0.0000 | [0.0000, 0.0000] | 1.0000 |
| 0.0100 | −0.4555 | [−0.5000, −0.4111] | 0.0098 | −0.9998 | [−0.9998, −0.9998] | 0.0098 |
| 0.0200 | −0.2556 | [−0.2889, −0.2222] | 0.0098 | −0.9999 | [−0.9999, −0.9999] | 0.0098 |

## 5.3 Primary contrasts and ranking ceiling

At a nominal budget of B = 0.001 the recall of Macro and Equal-FPR are the same. At B = 0.0025 there is a -0.0222 difference in recall. Which is not significantly different from zero. The recall for the Macro method is lower than Equal-FPR at larger budgets. Tail and Equal-FPR have a min client recall of zero until B = 0.005. When reaching B = 0.010 and B = 0.020 Equal-FPR can detect 0.9998 and 0.9999 min-client recall. But the Tail method still gets zero (see Table 2). The realized FPR for Equal-FPR is also higher at these budgets.

Reallocation of thresholds cannot reorder scores so the AUROC and AUPRC metrics remain the same for all policy methods on the same seed. The AUROC and AUPRC are calculated after concatenating held out benign scores of all 9 devices with their eligible attacks test scores. The AUROC is found using rank and the AUPRC is found by trapezoidal approximation. The averaged results over the 10 seeds give an AUROC of 0.9995 and AUPRC of 0.9999. This metric calculates the score separation on the devices using all their test rows so devices are not weighted equally. Nor are all attack types weighted equally in the data pool. The total data pool contains 1,041,078 attacks and 111,188 benign samples giving a prevalence of 90.35%. This will create higher baseline performance on the precision-recall curve. There is still a min client recall of zero for all policy budgets that fall under the FABRID family when looking at these results.

The CI and p-values are not controlling for a common FPR.

## 5.4 Secondary protocol stress test

For the secondary test the unrestricted form of macro is taken and normal local calibration is applied to the CIC IoT-DIAD dataset. Table 3 shows the average results over 10 scorers that were trained independently. There are 19 equally weighted clients that were trained using the reconstruction error as their anomaly score.

As seen in the primary experiment the federation FPR goes above the nominal value. At each increasing budget there are ratios of 9.80, 4.72, 3.20, 2.13 and 1.56. While the ratio decreases the measured FPR increases from 0.0098 to 0.0311. So the ratio going down doesn't mean that the FPR is decreasing. The secondary branch of this experiment does not include the Equal-FPR method. With the same scoring function and clients. This does not show if the Macro method outperforms the equal rate method at a comparable FPR.

**Table 3** CIC IoT-DIAD using unrestricted macro with local calibration. Scores are the reconstructed error and 19 equally weighted clients are shown over 10 seed averages.

| $B$ | Macro recall | Minimum-client recall | Realized FPR |
|---|---|---|---|
| 0.0010 | 0.3614 | 0.0000 | 0.0098 |
| 0.0025 | 0.3970 | 0.0000 | 0.0118 |
| 0.0050 | 0.4318 | 0.0007 | 0.0160 |
| 0.0100 | 0.4602 | 0.0089 | 0.0213 |
| 0.0200 | 0.5016 | 0.0251 | 0.0311 |

# 6 Discussion

It was not evaluated whether reallocation of thresholds can outperform the equal rate baseline at the same measured FPR.

When B = 0.001 using a conservative method of calibration allows the measured federation FPR to stay under the nominal budget. But the attack recall decreases. There are no attacks detected on the Danmini doorbell for the macro and tail methods. So none of these allocations can be said to perform well. They all fall under the nominal FPR.

Because attacks were used to estimate the utility function. The FABRID-IDS process requires knowledge of attacks to apply the allocation to a problem. It cannot select client rates without labeled attack examples.

The amount of time to solve for an allocation was measured separately from the other processes. From 95.7ms to 358.8ms for Macro and 310.3ms to 3883.1ms for Tail. No solvers had to fall back on the allocation calculations. These times are not a true measure of the latency. They are not including the anomaly score time or the communication of information to other clients and computing alerts.

More research is needed to find if the algorithm can be deployed with malicious clients. Privacy concerns and communication of alerts and whether the score function can be fixed to catch the attacks missed.

B = 0.020 shows how claiming the recall outperforms another method can be misleading. The Equal-FPR method is able to detect a macro recall of 1.0000 while running at a realized federation FPR of 1.88B. Whereas the Macro method detects 0.7444 while running at a FPR of 1.40B. So both methods are spending their nominal budget differently. It can only be concluded that these two methods work at these levels. If they were both measured at the same FPR it is not known which would be better.

This is a low power test because of using only 10 seeds. Which gives $2^{10} = 1024$ different signs that can be assigned. This leads to the smallest p-value being 0.0098 after using Holm correction. With 5 tests in the family. The smallest possible p-value is 0.001953125 if all signs were going in the same direction and no zeros. This was seen to happen for the min client recall tests at budget 0.010 and 0.020. Because the Tail method had zero minimum-client recall and the Equal-FPR had very little variation past 4 decimals. At B = 0.001 there is a zero-width CI and a p-value of one because the methods perform exactly the same. These p-values can only be corrected for within each family and the CI is for each point.

Eq. 3 can only give a 95% statement about one client's FPR. If the samples are i.i.d. then this can be applied to each client with at most a 5% chance that the population-FPR bound fails for each client. Applying a union bound to these nine clients only gives a lower bound of 1-9(.05)=.55 of hitting the thresholds on all nine clients. The data is not proven to meet these sample assumptions and no way can be found to state the federation FPR has a 95% rate. So it is not known if the N-BaIoT dataset will allow for the rates to hit these bounds.

Guaranteeing a certain false alarm rate would need tests on benign samples that are thought to represent the monitor. Then it can be seen if the false alarms match the expectations. This is something that needs to be done to monitor the rates as the system evolves.

The recall can change depending on the FPR rate. So it is important to keep that in mind when looking at the performance of the methods. Another factor is looking at the tail method's lower end of the client range. It can raise the performance on some of the worst clients but still have one client with no detections.

# 7 Conclusion

This paper proposed the FABRID-IDS process to allocate nominal FPR rates to the clients after training the detector. After the rates are picked the local threshold is found using the final calibration samples. The rates measured after training did not always fall under the allocated federation budget. When B = 0.001 on the N-BaIoT dataset the unrestricted versions of macro and tail had a measured federation FPR of .00170. When the rates were lowered with conservative calibration they were able to reach .00036. But lowered the attack recall. The Equal-FPR method detected more attacks at a few budget levels. But it was also higher on the measured FPR. It was not possible to find any positive values for the min client recall for any of the FABRID family of methods. When the process is applied to the CIC IoT-DIAD dataset there is a similar trend of the nominal FPR differing from the measured. Though there is no comparison of the equal rate method. Future work would like to see if the method can be applied while maintaining the false alarm rate in a monitored environment. Code and artifacts can be found at https://doi.org/10.5281/zenodo.22113172. Conflicts of Interest: The authors declare no conflict of interest.

# References


[1] Nguyen, T.D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., Sadeghi, A.-R.: DÏoT: A federated self-learning anomaly detection system for IoT. In: IEEE ICDCS, pp. 756–767 (2019). 10.1109/ICDCS.2019.00080

[2] Wang, H., Muñoz-González, L., Eklund, D., Raza, S.: Non-IID data re-balancing at IoT edge with peer-to-peer federated learning for anomaly detection. In: ACM WiSec, pp. 153–163 (2021). 10.1145/3448300.3467827

[3] Hamdi, N.: Federated learning-based intrusion detection system for Internet of Things. International Journal of Information Security **22**(6), 1937–1948 (2023). 10.1007/s10207-023-00727-6

[4] Laridi, S., Palmer, G., Tam, K.-M.M.: Enhanced federated anomaly detection through autoencoders using summary statistics-based thresholding. Scientific Reports **14**(1), 26704 (2024). 10.1038/s41598-024-76961-2

[5] Asiri, M., Khemakhem, M.A., Alhebshi, R.M., Alsulami, B.S., Eassa, F.E.: Decentralized federated learning for IoT malware detection at the multi-access edge: A two-tier, privacy-preserving design. Future Internet **17**(10), 475 (2025). 10.3390/fi17100475

[6] Khan, M.A., Khalid, O., Rais, R.N.B.: Fed-DTCN: A federated disentangled learning framework for unsupervised zero-day anomaly detection in IoT with semantic-aware augmentation. Sensors **26**(6), 1918 (2026). 10.3390/s26061918
[7] Kairouz, P., McMahan, H.B., *et al.*: Advances and open problems in federated learning. Foundations and Trends in Machine Learning **14**(1-2), 1–210 (2021). 10.1561/2200000083
[8] Bridges, R.A., Jamieson, J.D., Reed, J.W.: Setting the threshold for high throughput detectors: A mathematical approach for ensembles of dynamic, heterogeneous, probabilistic anomaly detectors. In: IEEE BigData, pp. 1071–1078 (2017). 10.1109/BigData.2017.8258031
[9] Ochiai, H., Nishihata, R., Tomiyama, E., Sun, Y., Esaki, H.: Detection of global anomalies on distributed IoT edges with device-to-device communication. In: MobiHoc '23, pp. 388–393 (2023). 10.1145/3565287.3616528
[10] Tong, X., Feng, Y., Li, J.J.: Neyman–Pearson classification algorithms and NP receiver operating characteristics. Science Advances **4**(2), eaao1659 (2018). 10.1126/sciadv.aao1659
[11] Angelopoulos, A.N., Bates, S.: Conformal prediction: A gentle introduction. Foundations and Trends in Machine Learning **16**(4), 494–591 (2023). 10.1561/2200000101
[12] Lari, E., Arablouei, R., Werner, S.: Communication-efficient byzantine-robust federated conformal prediction via partial sharing. IEEE Transactions on Signal Processing **74**, 4547–4563 (2026). 10.1109/tsp.2026.3737023
[13] Rockafellar, R.T., Uryasev, S.: Optimization of conditional value-at-risk. The Journal of Risk **2**(3), 21–41 (2000). 10.21314/jor.2000.038
[14] Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., Elovici, Y.: N-BaIoT: Network-based detection of IoT botnet attacks using deep autoencoders. IEEE Pervasive Computing **17**(3), 12–22 (2018). 10.1109/MPRV.2018.03367731
[15] Rabbani, M., Gui, J., Nejati, F., Zhou, Z., Kaniyamattam, A., Mirani, M., Piya, G., Opushnyev, I., Lu, R., Ghorbani, A.A.: Device identification and anomaly detection in IoT environments. IEEE Internet of Things Journal **12**(10), 13625–13643 (2025). 10.1109/JIOT.2024.3522863